

Lab 5 Istio Traffic Management 实验报告

学生姓名：姜丞骏 GitHub: whatokk

一、实验目的

本次实验围绕 Istio 的流量管理能力展开，主要完成 Request Routing、Traffic Shifting、Fault Injection 和 Circuit Breaking 四个主题的操作。实验基于已经部署完成的 Istio 与 Bookinfo 示例应用，通过配置 VirtualService、DestinationRule 等 Istio 资源，实现按用户请求路由、按权重灰度迁移、故障注入测试以及熔断保护。

通过本次实验，需要掌握以下内容：

1. 使用 VirtualService 将请求路由到指定服务版本。
2. 根据 HTTP 请求头中的用户信息实现差异化路由。
3. 使用权重规则将流量从一个服务版本逐步迁移到另一个服务版本。
4. 使用故障注入模拟延迟和错误响应。
5. 使用 DestinationRule 配置连接池和异常检测，实现熔断控制。
6. 使用 Fortio 对服务施加并发请求，验证熔断规则是否生效。
7. 将实验过程截图和说明整理为 Markdown 文档，并提交到 GitHub 仓库。

二、实验环境

项目	内容
操作系统	Windows 11
容器运行环境	Docker Desktop
Kubernetes 本地集群	Minikube
Kubernetes 版本	v1.32.8
Kubernetes 命令行工具	kubectl
Service Mesh	Istio 1.29.2
示例应用	Bookinfo、httpbin、fortio
报告格式	Markdown
提交平台	GitHub

本实验默认已经完成上一实验中的 Istio 安装、Bookinfo 部署、Gateway 配置以及 productpage 页面访问。如果环境尚未准备完成，需要先完成 Lab 4 的基础部署。

三、实验内容概述

本次实验按照 Istio 官方 Traffic Management 任务展开，主要包含以下四个部分：

1. Request Routing: 配置请求路由, 将不同用户的请求路由到不同版本的 reviews 服务。
2. Traffic Shifting: 配置权重路由, 将 reviews 服务流量从 v1 逐步迁移到 v3。
3. Fault Injection: 向 ratings 服务注入延迟和错误, 测试系统对故障的响应。
4. Circuit Breaking: 对 httpbin 服务配置熔断规则, 并使用 fortio 触发熔断。

实验整体流程如下:

```
检查 Kubernetes、Istio 与 Bookinfo 环境
↓
配置 Bookinfo 默认 DestinationRule
↓
Request Routing: 全部流量到 v1, jason 用户到 v2
↓
Traffic Shifting: reviews v1 与 v3 按权重迁移
↓
Fault Injection: 注入延迟与 HTTP 500 错误
↓
Circuit Breaking: 部署 httpbin 与 fortio, 配置熔断并验证
↓
整理截图与实验报告
```

四、实验准备

4.1 检查 Kubernetes、Istio 与 Bookinfo 状态

首先检查 minikube 集群、Istio 控制平面以及 Bookinfo 应用是否正常运行。

```
minikube status
kubectl get nodes
kubectl get pods -A
kubectl get pods -n istio-system
kubectl get services
kubectl get pods
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab5> minikube status
minikube
type: Control Plane
host: Running
kubelst: Running
apiserver: Running
kubeconfig: Configured

(base) PS D:\money\20260507\lab5> kubectl get nodes
NAME STATUS ROLES AGE VERSION
minikube Ready control-plane 3d21h v1.32.8
(base) PS D:\money\20260507\lab5> kubectl get pods -A
NAMESPACE NAME READY STATUS RESTARTS AGE
default bookinfo-gateway-istio-bf5f7ffc8-dznqv 1/1 Running 0 91m
default details-v1-766844796b-jtsfj 2/2 Running 0 96m
default productpage-v1-54bb874995-xfwm8 2/2 Running 0 96m
default ratings-v1-5dc79b6bcd-ccsgr 2/2 Running 0 96m
default reviews-v1-598b896c9d-6fftd 2/2 Running 0 96m
default reviews-v2-556d6457d-mwtlnl 2/2 Running 0 96m
default reviews-v3-564544b4d6-8pg4n 2/2 Running 0 96m
istio-system grafana-cdb9db549-nwxw4 1/1 Running 0 87m
istio-system istiod-547df67cfb-z4mdq 1/1 Running 1 (100m ago) 3d21h
istio-system jaeger-5d4bd98fc4-hc9m6 1/1 Running 0 87m
istio-system kiali-744c76b44c-c2n27 1/1 Running 0 87m
istio-system loki-0 2/2 Running 0 87m
istio-system prometheus-68fbbd698f-hrhw8 2/2 Running 0 87m
kube-system coredns-668d6bf9bc-z6qx6 1/1 Running 1 (100m ago) 3d21h
kube-system etcd-minikube 1/1 Running 1 (100m ago) 3d21h
kube-system kube-apiserver-minikube 1/1 Running 1 (100m ago) 3d21h
kube-system kube-controller-manager-minikube 1/1 Running 1 (100m ago) 3d21h
kube-system kube-proxy-9bzg5 1/1 Running 1 (100m ago) 3d21h
kube-system kube-scheduler-minikube 1/1 Running 1 (100m ago) 3d21h
kube-system storage-provisioner 1/1 Running 1 (100m ago) 3d21h
(base) PS D:\money\20260507\lab5> kubectl get pods -n istio-system
NAME READY STATUS RESTARTS AGE
grafana-cdb9db549-nwxw4 1/1 Running 0 87m
istiod-547df67cfb-z4mdq 1/1 Running 1 (100m ago) 3d21h
jaeger-5d4bd98fc4-hc9m6 1/1 Running 0 87m
kiali-744c76b44c-c2n27 1/1 Running 0 87m
loki-0 2/2 Running 0 87m
prometheus-68fbbd698f-hrhw8 2/2 Running 0 87m
(base) PS D:\money\20260507\lab5> kubectl get services
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
bookinfo-gateway-istio ClusterIP 10.102.160.106 <none> 15021/TCP, 80/TCP 91m
details ClusterIP 10.105.27.77 <none> 9080/TCP 96m
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 3d21h
productpage ClusterIP 10.106.188.204 <none> 9080/TCP 96m
ratings ClusterIP 10.102.136.227 <none> 9080/TCP 96m
reviews ClusterIP 10.103.137.70 <none> 9080/TCP 96m
(base) PS D:\money\20260507\lab5> kubectl get pod
NAME READY STATUS RESTARTS AGE
bookinfo-gateway-istio-bf5f7ffc8-dznqv 1/1 Running 0 91m
details-v1-766844796b-jtsfj 2/2 Running 0 96m
productpage-v1-54bb874995-xfwm8 2/2 Running 0 96m
ratings-v1-5dc79b6bcd-ccsgr 2/2 Running 0 96m
reviews-v1-598b896c9d-6fftd 2/2 Running 0 96m
reviews-v2-556d6457d-mwtlnl 2/2 Running 0 96m
reviews-v3-564544b4d6-8pg4n 2/2 Running 0 96m
```

环境状态检查

从截图可以看到，minikube 集群处于 Running 状态，节点状态为 Ready；Istio 控制平面组件正常运行；Bookinfo 应用中的 productpage、details、ratings、reviews 等服务均已部署完成。

4.2 配置 Bookinfo 默认 DestinationRule

在使用 Istio 版本路由之前，需要为 Bookinfo 服务定义不同版本对应的 subset。

```
kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
kubectl get destinationrules
kubectl get destinationrule reviews -o yaml
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab5\istio-1.29.2> kubectl apply -f samples/bookinfo/networking/destination-
rule-all.yaml
>> kubectl get destinationrules
>> kubectl get destinationrule reviews -o yaml
destinationrule.networking.istio.io/productpage unchanged
destinationrule.networking.istio.io/reviews unchanged
destinationrule.networking.istio.io/ratings unchanged
destinationrule.networking.istio.io/details unchanged
NAME          HOST          AGE
details       details       4d10h
productpage   productpage   4d10h
ratings       ratings       4d10h
reviews       reviews       4d10h
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"DestinationRule","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":{"host":"reviews","subsets":[{"labels":{"version":"v1"},"name":"v1"}, {"labels":{"version":"v2"},"name":"v2"}, {"labels":{"version":"v3"},"name":"v3"}]}}
  creationTimestamp: "2026-05-13T04:41:09Z"
  generation: 1
  name: reviews
  namespace: default
  resourceVersion: "15511"
  uid: aee83fe6-ece3-427b-8ffd-858b8689ce8a
spec:
  host: reviews
  subsets:
  - labels:
    version: v1
    name: v1
  - labels:
    version: v2
    name: v2
  - labels:
    version: v3
    name: v3
```

Bookinfo DestinationRule

从截图可以看到，Bookinfo 的各个服务已经创建 DestinationRule，并为不同版本定义了 subset。例如 reviews 服务包含 v1、v2、v3 三个版本，后续 VirtualService 可以根据这些 subset 进行路由控制。

五、Request Routing 实验

5.1 将所有服务流量路由到 v1

首先将 Bookinfo 中所有服务的流量都路由到 v1 版本。

```
kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
kubectl get virtualservices
kubectl get virtualservice reviews -o yaml
```

执行后访问 productpage 页面。

```
kubectl port-forward svc/bookinfo-gateway-istio 8080:80
```

浏览器访问：

<http://localhost:8080/productpage>

实验结果如下图所示。

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Book Reviews

"An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!"



Reviewer1
Reviews served by: reviews-v1-598b896c9d-6fftd

"Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare."



Reviewer2
Reviews served by: reviews-v1-598b896c9d-6fftd

Request Routing 全部流量到 v1

从截图可以看到，VirtualService 已经将 reviews 服务流量路由到 subset v1。页面中的 reviews 区域不显示星级，这是因为 reviews:v1 不会调用 ratings 服务。

5.2 配置 jason 用户路由到 reviews v2

接下来配置基于用户身份的路由规则，使用户 jason 的请求路由到 reviews:v2，其他用户仍然路由到 reviews:v1。

```
kubect1 apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
kubect1 get virtualservice reviews -o yaml
```

在 productpage 页面中登录用户：

jason

然后刷新页面。

实验结果如下图所示。

The Comedy of Errors

Wikipedia Summary: The Comedy of Errors is one of **William Shakespeare's** early plays. It is his shortest and one of his most farcical comedies, with a major part of the humour coming from slapstick and mistaken identity, in addition to puns and word play.

[Learn more about Istio](#) →

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Book Reviews

★★★★★

"An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!"



Reviewer1

Reviews served by: reviews-v2-556d6457d-mwtn1

★★★★☆

"Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare."



Reviewer2

Reviews served by: reviews-v2-556d6457d-mwtn1

jason 用户路由到 reviews v2

从截图可以看到，VirtualService 中根据请求头 `end-user: jason` 进行匹配。用户 `jason` 登录后，`reviews` 区域显示星级评分，说明该用户请求已经被路由到 `reviews:v2`。其他用户或未登录用户仍然访问 `reviews:v1`。

六、Traffic Shifting 实验

6.1 将 50% 流量迁移到 reviews v3

为了观察流量迁移效果，先确保 `reviews` 流量处于 `v1` 初始状态。

```
kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
```

随后执行以下命令，将 `reviews` 服务的 50% 流量路由到 `v1`，另外 50% 流量路由到 `v3`。

```
kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-50-v3.yaml  
kubectl get virtualservice reviews -o yaml
```

多次刷新 `productpage` 页面。

实验结果如下图所示。

从截图可以看到，VirtualService 中 reviews:v1 和 reviews:v3 的权重均为 50。多次刷新页面时，reviews 区域有时不显示星级，有时显示红色星级，说明流量被按权重分配到两个不同版本。

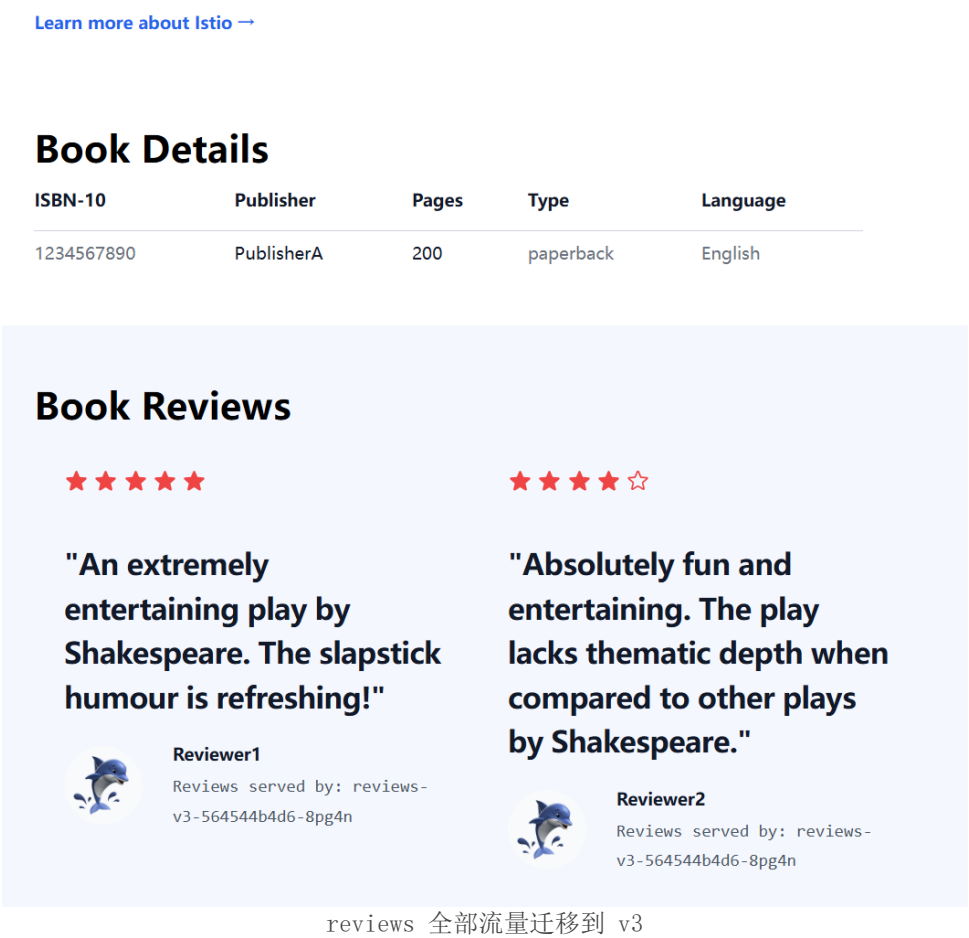
6.2 将 100% 流量迁移到 reviews v3

继续执行以下命令，将 reviews 服务的全部流量迁移到 v3。

```
kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-v3.yaml
kubectl get virtualservice reviews -o yaml
```

刷新 productpage 页面。

实验结果如下图所示。



从截图可以看到，reviews 服务的全部流量已经路由到 reviews:v3。页面中稳定显示红色星级评分，说明流量迁移已经完成。

七、Fault Injection 实验

7.1 注入 HTTP 延迟故障

Fault Injection 实验需要先让 jason 用户进入 reviews:v2，其他用户进入 reviews:v1。

```
kubectI apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
kubectI apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
```

接下来对用户 jason 从 reviews:v2 到 ratings 服务的调用注入 7 秒延迟。

```
kubectI apply -f samples/bookinfo/networking/virtual-service-ratings-test-delay.yaml
kubectI get virtualservice ratings -o yaml
```

在 productpage 页面登录 jason 并刷新页面。

实验结果如下图所示。

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Error fetching product reviews

Sorry, product reviews are currently unavailable for this book.

ratings 延迟故障注入

从截图可以看到，ratings 服务针对请求头 end-user: jason 注入了 fixedDelay: 7s 的延迟故障。页面中 reviews 区域出现不可用提示，这是因为 productpage 到 reviews 的调用存在较短超时限制，导致页面提前超时并显示错误信息。

7.2 注入 HTTP Abort 故障

接下来将延迟故障改为 HTTP Abort 故障，使用户 jason 调用 ratings 服务时返回 HTTP 500。

```
kubectI apply -f samples/bookinfo/networking/virtual-service-ratings-test-abort.yaml
kubectI get virtualservice ratings -o yaml
```

在浏览器中继续使用用户 jason 刷新 productpage 页面。

实验结果如下图所示。

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Book Reviews

Ratings service is currently unavailable

"An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!"



Reviewer1

Reviews served by: reviews-v2-556d6457d-mwtn1

Ratings service is currently unavailable

"Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare."



Reviewer2

Reviews served by: reviews-v2-556d6457d-mwtn1

ratings HTTP Abort 故障注入

从截图可以看到，ratings 服务针对 jason 用户被注入 HTTP 500 错误，页面中 ratings 相关内容出现不可用提示。使用其他用户访问时，由于其他用户走 reviews:v1，不调用 ratings 服务，因此不会出现相同错误。

八、Circuit Breaking 实验

Circuit Breaking 实验不使用 Bookinfo，而是使用 Istio 官方提供的 httpbin 服务作为后端，使用 fortio 作为压测客户端。

8.1 部署 httpbin 与 fortio

为了避免前面故障注入规则影响后续观察，可以先删除 ratings 的 VirtualService。

```
kubectl delete virtualservice ratings --ignore-not-found
```

部署 httpbin 服务。

```
kubectl apply -f samples/httpbin/httpbin.yaml
kubectl get pods -l app=httpbin
kubectl get svc httpbin
```

部署 fortio 客户端。

```
kubectl apply -f samples/httpbin/sample-client/fortio-deploy.yaml
```

```
kubectl get pods -l app=fortio
```

设置 PowerShell 变量保存 fortio Pod 名称。

```
$FORTIO_POD = kubectl get pods -l app=fortio -o jsonpath="{.items[0].metadata.name}"  
echo $FORTIO_POD
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab5\istio-1.29.2> kubectl get pods -l app=fortio  
NAME                                READY   STATUS    RESTARTS   AGE  
fortio-deploy-5dcb7c586c-254qg      2/2     Running   0           3m38s  
(base) PS D:\money\20260507\lab5\istio-1.29.2> kubectl get svc httpbin  
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE  
httpbin   ClusterIP   10.98.65.255 <none>        8000/TCP    3m54s
```

httpbin 与 fortio 状态

从截图可以看到，httpbin 服务和 fortio 客户端已经部署成功。fortio Pod 若显示 2/2 Running，说明业务容器与 Istio Sidecar 均已正常启动。

8.2 配置 httpbin 熔断规则

创建 DestinationRule，为 httpbin 服务配置连接池和异常检测规则。

```
@&quot;  
apiVersion: networking.istio.io/v1  
kind: DestinationRule  
metadata:  
  name: httpbin  
spec:  
  host: httpbin  
  trafficPolicy:  
    connectionPool:  
      tcp:  
        maxConnections: 1  
      http:  
        http1MaxPendingRequests: 1  
        maxRequestsPerConnection: 1  
    outlierDetection:  
      consecutive5xxErrors: 1  
      interval: 1s  
      baseEjectionTime: 3m  
      maxEjectionPercent: 100  
&quot;@ | kubectl apply -f -
```

查看 DestinationRule。

```
kubectl get destinationrule httpbin -o yaml
```

实验结果如下图所示。

```

(base) PS D:\money\20260507\lab5\istio-1.29.2> kubectl get destinationrule httpbin -o yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"DestinationRule","metadata":{"annotations":{},"name":"httpbin","namespace":"default"},"spec":{"host":"httpbin","trafficPolicy":{"connectionPool":{"http":{"http1MaxPendingRequests":1,"maxRequestsPerConnection":1},"tcp":{"maxConnections":1}},"outlierDetection":{"baseEjectionTime":"3m","consecutive5xxErrors":1,"interval":"1s","maxEjectionPercent":100}}}}
  creationTimestamp: "2026-05-17T15:40:43Z"
  generation: 1
  name: httpbin
  namespace: default
  resourceVersion: "60161"
  uid: a519b004-bf4d-4dfb-a6c4-fa5960e2fc21
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
      tcp:
        maxConnections: 1
    outlierDetection:
      baseEjectionTime: 3m
      consecutive5xxErrors: 1
      interval: 1s
      maxEjectionPercent: 100

```

httpbin 熔断规则

从截图可以看到，DestinationRule 对 httpbin 服务配置了最大连接数、最大等待请求数、每连接最大请求数以及异常检测策略。该规则用于在高并发请求或异常请求过多时触发熔断。

8.3 使用 fortio 触发熔断

先发送一次普通请求，确认 fortio 可以正常访问 httpbin。

```
kubectl exec "${FORTIO_POD}" -c fortio -- /usr/bin/fortio curl -quiet http://httpbin:8000/get
```

然后使用 fortio 发送并发请求。

```
kubectl exec "${FORTIO_POD}" -c fortio -- /usr/bin/fortio load -c 2 -qps 0 -n 20 -loglevel Warning http://httpbin:8000/get
```

继续提高并发数。

```
kubectl exec "${FORTIO_POD}" -c fortio -- /usr/bin/fortio load -c 3 -qps 0 -n 30 -loglevel Warning http://httpbin:8000/get
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab5\istio-1.29.2> kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio 1
oad -c 3 -qps 0 -n 30 -loglevel Warning http://httpbin:8000/get
{"ts":1779032504.043039,"level":"info","r":1,"file":"logger.go","line":298,"msg":"Log level is now 3 War
ning (was 2 Info)"}
Fortio 1.69.5 running at 0 queries per second, 16->16 procs, for 30 calls: http://httpbin:8000/get
Starting at max qps with 3 thread(s) [gomax 16] for exactly 30 calls (10 per thread + 0)
{"ts":1779032504.054592,"level":"warn","r":8,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1779032504.055142,"level":"warn","r":6,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1779032504.055415,"level":"warn","r":7,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1779032504.058384,"level":"warn","r":7,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1779032504.058406,"level":"warn","r":8,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1779032504.058969,"level":"warn","r":6,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1779032504.062760,"level":"warn","r":6,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1779032504.063302,"level":"warn","r":7,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1779032504.063390,"level":"warn","r":8,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1779032504.066143,"level":"warn","r":8,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1779032504.066611,"level":"warn","r":7,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1779032504.066689,"level":"warn","r":6,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1779032504.068987,"level":"warn","r":6,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1779032504.069540,"level":"warn","r":8,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1779032504.069562,"level":"warn","r":7,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1779032504.071926,"level":"warn","r":6,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1779032504.072372,"level":"warn","r":8,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1779032504.072416,"level":"warn","r":7,"file":"http_client.go","line":1151,"msg":"Non ok http code
"code":503,"status":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1779032504.090387,"level":"warn","r":6,"file":"http_client.go","line":1151,"msg":"Non ok http code
```

fortio 并发请求触发熔断

从截图可以看到，当并发请求增加后，部分请求返回 HTTP 503。说明当并发请求超过 DestinationRule 中设置的连接池限制时，Istio Sidecar 会触发熔断，拒绝部分请求。

8.4 查看 istio-proxy 熔断统计信息

查看 fortio Pod 中 istio-proxy 的统计信息。

```
kubectl exec &quot;$FORTIO_POD&quot; -c istio-proxy -- pilot-agent request GET stats | Select-
String &quot;httpbin&quot; | Select-String &quot;pending&quot;
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab5\istio-1.29.2> kubectl exec "$FORTIO_POD" -c istio-proxy -- pilot-agent
request GET stats | Select-String "httpbin" | Select-String "pending"

cluster.outbound|8000|httpbin.default.svc.cluster.local;.circuit_breakers.default.rq_pending_open: 0
cluster.outbound|8000|httpbin.default.svc.cluster.local;.circuit_breakers.high.rq_pending_open: 0
cluster.outbound|8000|httpbin.default.svc.cluster.local;.upstream_rq_pending_active: 0
cluster.outbound|8000|httpbin.default.svc.cluster.local;.upstream_rq_pending_failure_eject: 0
cluster.outbound|8000|httpbin.default.svc.cluster.local;.upstream_rq_pending_overflow: 0
cluster.outbound|8000|httpbin.default.svc.cluster.local;.upstream_rq_pending_total: 4
```

istio-proxy 熔断统计

从输出可以看到与 httpbin 相关的 pending overflow 统计信息，例如 upstream_rq_pending_overflow。该字段表示由于请求排队溢出而被熔断的请求数量。出现该统计值说明熔断规则已经生效。

九、实验结果分析

本次实验完成了 Istio Traffic Management 的四类典型能力验证。

Request Routing 实验表明，Istio 可以通过 VirtualService 将服务流量路由到指定版本，也可以根据 HTTP 请求头中的用户信息将特定用户路由到特定服务版本。实验中，普通用户访问 reviews:v1，而用户 jason 被路由到 reviews:v2。

Traffic Shifting 实验表明，Istio 可以通过权重规则控制不同版本服务的流量比例。实验中，reviews 服务先全部流量进入 v1，随后配置为 v1 与 v3 各 50%，最后全部迁移到 v3。该过程模拟了微服务版本灰度发布和迁移过程。

Fault Injection 实验表明，Istio 可以主动向服务调用链路注入延迟或错误，用于测试系统容错能力。实验中，对用户 jason 调用 ratings 服务时注入 7 秒延迟，导致 productpage 到 reviews 的调用提前超时，从而暴露出微服务之间超时配置不一致的问题。之后注入 HTTP 500 错误，验证了应用在下游服务异常时的表现。

Circuit Breaking 实验表明，Istio 可以通过 DestinationRule 配置连接池和异常检测策略，对高并发请求进行限制。实验中，当 fortio 使用较高并发访问 httpbin 时，部分请求返回 HTTP 503，并且 istio-proxy 统计信息中出现 pending overflow，说明熔断规则已经生效。

十、实验中遇到的问题与解决方法

10.1 PowerShell 与 Linux shell 命令不同

Istio 官方文档中部分命令使用 Linux shell 语法，例如 export、grep 和 here document。Windows PowerShell 中需要改写为 PowerShell 语法。

Linux 中设置变量的写法为：

```
export FORTIO_POD=$(kubectl get pods -l app=fortio -o &#x27;jsonpath={. items[0].metadata.name}&#x27;)
```

PowerShell 中应写为：

```
$FORTIO_POD = kubectl get pods -l app=fortio -o jsonpath=&quot;{. items[0].metadata.name}&quot;;
```

Linux 中使用 grep 过滤输出，PowerShell 中可以使用 Select-String。

```
kubectl exec &quot;${FORTIO_POD}&quot; -c istio-proxy -- pilot-agent request GET stats | Select-String &quot;httpbin&quot;; | Select-String &quot;pending&quot;;
```

10.2 Bookinfo 页面访问需要保持 port-forward 窗口开启

访问 `productpage` 页面时，需要保持以下命令所在的 PowerShell 窗口开启。

```
kubectl port-forward svc/bookinfo-gateway-istio 8080:80
```

如果关闭该窗口，浏览器将无法继续访问：

```
http://localhost:8080/productpage
```

10.3 VirtualService 配置存在传播延迟

应用新的 `VirtualService` 后，Istio 配置需要短时间传播到各个 `Sidecar`。若页面效果没有立即变化，可以等待数秒后刷新页面。

10.4 故障注入只对 `jason` 用户生效

`Fault Injection` 实验中的延迟和 `abort` 规则匹配的是请求头 `end-user: jason`。因此必须先在 `productpage` 页面登录为 `jason`，否则无法观察到对应故障效果。

10.5 熔断实验需要确认 `fortio` Pod 注入 `Sidecar`

`Circuit Breaking` 实验依赖 `fortio` Pod 中的 Istio `Sidecar` 执行流量控制。若 `fortio` Pod 没有显示 `2/2 Running`，需要确认 `default` 命名空间是否启用了自动注入。

```
kubectl get namespace default --show-labels
kubectl label namespace default istio-injection=enabled --overwrite
```

十一、清理实验资源

如果后续不再继续实验，可以清理本次实验创建的规则和示例服务。

清理 `Bookinfo` 相关路由规则。

```
kubectl delete virtualservice reviews ratings details productpage --ignore-not-found
kubectl delete destinationrule reviews ratings details productpage --ignore-not-found
```

清理 `httpbin` 和 `fortio`。

```
kubectl delete destinationrule httpbin --ignore-not-found
kubectl delete -f samples/httpbin/sample-client/fortio-deploy.yaml
kubectl delete -f samples/httpbin/httpbin.yaml
```

如果需要完全清理 `Bookinfo`，可以执行：

```
kubectl delete -f samples/bookinfo/platform/kube/bookinfo.yaml
```

如果需要停止 `minikube`，可以执行：

```
minikube stop
```

十二、实验总结

本次实验完成了 Istio 流量管理中的四个核心任务，包括请求路由、流量迁移、故障注入和熔断控制。

通过 Request Routing 实验，可以看到 Istio 能够根据服务版本和请求头信息进行精细化路由控制。通过 Traffic Shifting 实验，可以看到 Istio 能够按权重逐步迁移服务流量，适合用于灰度发布和版本迁移。通过 Fault Injection 实验，可以主动制造延迟和错误，观察微服务系统在异常情况下的表现，从而发现服务间超时配置不一致等问题。通过 Circuit Breaking 实验，可以限制服务连接和请求并发，防止异常流量进一步扩散。

整体来看，Istio 的 Traffic Management 能力可以在不修改业务代码的情况下，对微服务之间的通信行为进行控制。它将路由、灰度、故障测试和熔断等能力下沉到服务网格层，为微服务架构中的版本发布、故障演练和系统保护提供了统一手段。

十三、参考资料

1. Istio Request Routing
<https://istio.io/latest/docs/tasks/traffic-management/request-routing/>
2. Istio Traffic Shifting
<https://istio.io/latest/docs/tasks/traffic-management/traffic-shifting/>
3. Istio Fault Injection
<https://istio.io/latest/docs/tasks/traffic-management/fault-injection/>
4. Istio Circuit Breaking
<https://istio.io/latest/docs/tasks/traffic-management/circuit-breaking/>